

Privacy-aware Environment Map for Care-O-bot 4

Nirish Samant [NS]
Institute of Computer Science
University of Göttingen
Göttingen, Germany
nirish.samant
@stud.uni-goettingen.de

Naveen Parimireddy [NP]
Institute of Computer Science
University of Göttingen
Göttingen, Germany
n.parimireddy
@stud.uni-goettingen.de

Abstract—Service robots deployed in indoor environments increasingly operate in spaces that contain privacy-sensitive objects. These robots must not only navigate safely but also remain aware of privacy-sensitive elements within their surroundings. This paper presents the design and implementation of a privacy-aware environment mapping system using the Care-O-Bot 4 service robot in a simulated environment. The system integrates LiDAR-based Simultaneous Localization and Mapping (SLAM) with camera-based object detection using YOLOv8 to construct a 2D occupancy grid map of the environment while simultaneously identifying privacy-sensitive objects within it. Laser scan data is processed using the gmapping SLAM package to generate the environment map, while a YOLO-based detector analyzes the robot’s RGB camera feed to recognize privacy-relevant objects. Detected objects are then localized in 3D using the robot’s depth camera and projected onto the map as visual markers in RViz, producing a unified representation that combines spatial and privacy-relevant semantic information. The proposed approach demonstrates a practical method for embedding privacy awareness directly into a robot’s spatial understanding of its environment, offering a foundation for more privacy-conscious autonomous navigation in shared human spaces.

Index Terms—Privacy-aware robotics, Care-O-Bot, Gazebo simulation, SLAM, YOLOv8, Object detection, Environment mapping

I. INTRODUCTION

Robotic systems are increasingly being used in indoor environments such as homes, offices, hospitals, and assisted-living facilities. In these environments, robots need to perceive and understand their surroundings in order to navigate safely and perform their tasks. At the same time, the environments in which robots operate may contain objects that are sensitive from a privacy perspective. A robot equipped with cameras and sensors may inadvertently capture or expose sensitive information present in the environment. Therefore, a robotic system should not only understand the spatial structure of its environment but should also be capable of identifying and representing privacy-sensitive objects.

The objective of this project, *Creating a Privacy-aware Environment Map for Care-O-bot 4*, is to develop a system that combines environmental mapping with privacy-sensitive object detection. The project is implemented using the Care-O-Bot platform and its simulated environment. The system uses LiDAR data to construct a two-dimensional map of the environment while a camera is used to identify objects that may be relevant from a privacy perspective.

For environment mapping, LiDAR measurements are processed using a Simultaneous Localization and Mapping (SLAM) approach. The LiDAR data is used with the GMapping algorithm to generate an occupancy map of the simulated environment. The resulting map can be visualized together with the robot and sensor data using RViz.

In parallel with the mapping process, the camera provides image data for detecting privacy-sensitive objects. A YOLO-based object detection node processes the camera images and publishes the detected objects. However, image-based detections initially provide information in the camera image coordinate system and therefore cannot directly be placed on the environment map. To address this problem, depth information and coordinate transformations are used to estimate the position of each detected object in the environment. The detected object, its corresponding depth information, and the relevant coordinate transformations are combined to obtain a position in the map coordinate system. These positions are then published and represented as visual markers in RViz.

The resulting system therefore combines two complementary perception processes: LiDAR-based mapping for understanding the spatial structure of the environment and camera-based object detection for identifying privacy-sensitive objects. By integrating the detected objects with the generated map, the system provides a privacy-aware representation of the environment. This approach forms the basis for further development of robotic systems that can reason about both spatial information and privacy-related aspects of their surroundings.

The remainder of this paper is structured as follows. Section II reviews related work on SLAM, object detection, semantic mapping and privacy in robotics. Section III describes the implemented system design and architecture. Section IV details the implementation of each pipeline component. Section V presents results obtained to date. Section VI discusses limitations and future directions. Section VII concludes the paper.

II. RELATED WORK

This section reviews prior work in three areas relevant to the project: Simultaneous Localization and Mapping (SLAM) for mobile robots, object detection for robotic perception, and the emerging area of semantic and privacy-aware mapping.

A. Simultaneous Localization and Mapping (SLAM)

SLAM is a foundational problem in mobile robotics, enabling a robot to construct a map of an unknown environment while simultaneously estimating its own pose within that map [1]. Early work by Moreno et al. [2] demonstrated that combining LiDAR-based mapping with vision-based object recognition allows a robot to build a map that links object detections to 3D positions.

Laser-based 2D SLAM approaches, such as the Rao-Blackwellized particle filter method used in `gmapping` [3], remain widely used for indoor mobile robots due to their reliability and relatively low computational cost compared to full 3D SLAM systems. These approaches rely on accurate odometry and consistent transform (TF) frames between the robot's base, sensors, and the map frame; misconfiguration of these frames, as encountered during this project, is a commonly reported source of mapping failure [4]. More recent SLAM systems (e.g., Cartographer [5], ORB-SLAM [6]) extend these ideas to graph-based optimization and visual or visual-inertial sensing, though classical laser-based SLAM remains sufficient for structured indoor environments such as the one used in this project.

B. Object Detection for Robotic Perception

Deep learning-based object detectors have become the standard approach for enabling robots to semantically interpret their camera feeds. Single-stage detectors such as the YOLO (You Only Look Once) family [7], [8] offer real-time detection performance suitable for onboard robotic use, trading a small amount of accuracy for significantly reduced inference latency compared to two-stage detectors such as Faster R-CNN [9]. YOLO models have been widely adopted in robotics for tasks including object recognition, grasp detection, and scene understanding, due to their ease of integration with ROS via lightweight Python nodes.

More recent approaches leverage deep learning for detection; Salas-Moreno et al. [10] proposed SLAM++ which integrates object recognition directly into the SLAM loop, though this approach requires known object models.

C. Semantic and Privacy-Aware Mapping

In the context of privacy, relatively little work directly addresses the problem of mapping privacy-sensitive locations. Birchfield and Konolige [11] discuss the challenge of robots operating in human-populated environments and the need for spatial awareness of sensitive areas. Das et al. [12] propose a framework for robots to respect privacy zones in smart home environments, treating privacy as a spatial constraint on robot behavior.

Beyond geometric occupancy grids, there is growing interest in *semantic mapping*, where detected objects and their classes are embedded into the spatial map rather than treated as separate perception outputs [13]. Semantic maps allow robots to reason about *what* exists in a location, not just *whether* that location is occupied. Building on this idea, a smaller body

of work has begun to explore privacy-aware robotic perception and mapping, motivated by the increasing deployment of robots in private, human-occupied spaces such as homes and offices [14]. Such work highlights the need for robots to recognize and appropriately handle sensitive information encountered during operation, whether by avoiding capture, anonymizing sensor data, or (as in this project) explicitly flagging sensitive regions within the map for downstream awareness or avoidance behaviors.

D. Positioning of This Work

This project builds on the intersection of these three areas: it uses a classical particle-filter SLAM approach (`gmapping`) for environment mapping, integrates a YOLO-based detector for identifying privacy-sensitive objects, and combines the two into a semantic, privacy-aware map representation using RViz marker visualization. Unlike much of the semantic mapping literature, which focuses on general object categories for navigation or manipulation, this work specifically targets privacy-sensitive object classes, positioning it within the smaller but growing body of work on privacy-conscious robotic perception.

III. SYSTEM DESIGN AND ARCHITECTURE

This section describes the software stack, robot platform, simulation environment and architecture used to implement the privacy-aware environment mapping system.

A. Software Stack

The system was implemented using ROS Noetic with a Catkin workspace and Gazebo Classic for Care-O-Bot simulation. GMapping was used for SLAM, while YOLO was used for privacy-sensitive object detection. Custom Python ROS nodes handled object localization and RViz visualization. TF was used to transform detected object positions between camera, LiDAR, robot, and map coordinate frames.

B. Robot Platform and Simulation Environment

The Care-O-Bot 4 is a mobile service robot featuring an omnidirectional base, a torso-mounted arm, and multiple on-board sensors including front-facing LiDAR, a head-mounted camera system etc. All experiments are conducted within the Gazebo 11 simulator under ROS Noetic on Ubuntu 20.04.

The Care-O-Bot simulator (see figure 1) provides the robot model, simulated sensors, robot state, and indoor environment required for the project. The simulated environment allows the robot to be moved through the environment while sensor measurements are generated continuously. This provides a controlled environment in which the mapping and privacy-detection components can be developed and tested.

The simulation environment represents a typical indoor apartment space. Privacy-sensitive objects are represented by proxy models from the Gazebo default model database. These object categories were selected because they are available in the Gazebo offline model database and have corresponding detection classes in the COCO dataset used by YOLOv8.

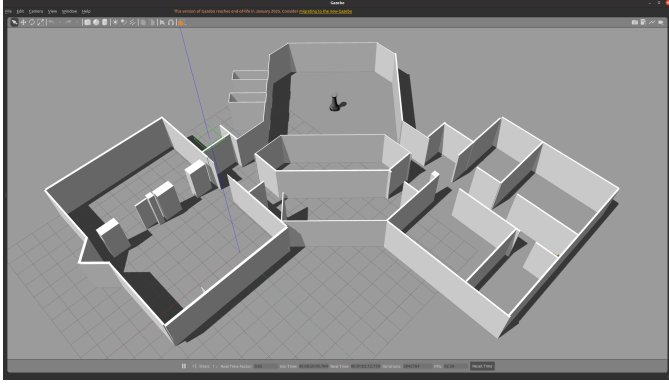


Fig. 1. Gazebo simulation environment representing an indoor apartment layout used for Care-O-Bot navigation, SLAM mapping, and privacy-sensitive object detection experiments.

C. Overall Architecture

The complete system can be divided into two main perception pipelines. The first pipeline is responsible for constructing the geometric representation of the environment, while the second pipeline identifies and localizes privacy-sensitive objects. The outputs of both pipelines are subsequently combined to produce the final privacy-aware environment map.

The overall data flow can be summarized as follows:

LiDAR → SLAM → Environment Map

**Camera → Object Detection → Depth Information →
Coordinate Transformation → Map Position → Privacy
Marker in Environment Map.**

IV. IMPLEMENTATION

A. Robot Sensors Used

The Care-O-Bot platform provides several sensors that are used in the privacy-aware mapping system. In the implemented configuration, the following sensors and data sources were used:

- **LiDAR:** The Care-O-Bot provides three planar laser scanners, whose measurements are combined into the unified `/scan_unified` topic. As summarized in Table I, this unified laser scan is used as the laser input to the SLAM system for environment mapping.
- **RGB cameras:** Privacy-sensitive objects are detected from the RGB image streams of the torso-mounted 3D cameras using the YOLO-based `privacy_detector.py` node. Two detector instances are executed in parallel, using the left and right RGB feeds listed in Table I, respectively. For each detection, the detector provides the object class, confidence score, bounding-box coordinates, image timestamp, and source camera identifier.

- **RGB-D depth sensors:** The metric 3D position of each detected privacy-sensitive object is obtained from the registered depth streams corresponding to the RGB cameras, as listed in Table I. The center of each detected bounding box is associated with the corresponding pixel in the registered depth image. A local depth patch is sampled to obtain a valid metric depth measurement, which is then back-projected using the camera intrinsics from the corresponding `CameraInfo` topic. This produces a 3D point in the camera optical frame.
- **TF and robot pose:** The camera-frame 3D point is transformed into the global map frame using the TF transformations listed in Table I. The transformation between map and `base_link` is additionally used to obtain the robot pose and calculate the planar distance between the robot and the localized privacy-sensitive object. The SLAM system uses `/scan_unified` together with the configured `base_link` and `odom_combined` frames to maintain the robot's global pose.

B. Pipeline Architecture

The system is organized as a four-layer pipeline, illustrated conceptually as follows:

- 1) **Simulation Layer:** Gazebo environment with Care-O-Bot and spawned privacy-sensitive objects.
- 2) **LiDAR-Based Mapping Layer:** LiDAR SLAM using GMapping to generate a 2D occupancy grid map.
- 3) **Object Detection Layer:** YOLOv8-based object detection on camera frames to identify privacy-sensitive objects and their bounding boxes.
- 4) **Privacy Marker Layer:** Projection of 2D detections into 3D world coordinates using camera intrinsics and depth estimation, annotating detected object locations on the occupancy map.

C. LiDAR-Based Mapping Layer

The first processing branch uses LiDAR measurements to construct a two-dimensional representation of the environment (see figure 2). The LiDAR data is provided through the `/scan_unified` topic. The laser measurements are supplied to the GMapping SLAM node, which uses the sensor observations together with robot motion information to estimate the robot pose and construct an occupancy map.

Mapping was performed using the `slam_gmapping` node, which implements a Rao-Blackwellized particle filter. GMapping was configured and launched with the following key parameters to accommodate the Care-O-Bot's non-standard frame identifiers:

```
roslaunch slam_gmapping \
  scan:=/base_laser_front/scan \
  _odom_frame:=odom_combined
```

The robot was then driven manually through the environment using the `teleop_twist_keyboard` package, with velocity commands remapped to the Care-O-Bot's navigation topic:

TABLE I
ROBOT SENSOR FEEDS AND ROS DATA SOURCES USED IN THE SYSTEM.

Sensor / Source	ROS topic / frame	Purpose
Planar LiDAR	/base_laser_front/scan, /base_laser_left/scan, /base_laser_right/scan; /scan_unified	Laser measurements used by the SLAM system for environment mapping.
Left RGB camera	/torso_cam3d_left/rgb/image_raw	RGB image input to the left YOLO detector for privacy-sensitive object detection.
Right RGB camera	/torso_cam3d_right/rgb/image_raw	RGB image input to the right YOLO detector for privacy-sensitive object detection.
Left RGB-D depth	/torso_cam3d_left/depth_registered	Registered depth corresponding to the left RGB image. Used to obtain the metric depth of detected objects.
Right RGB-D depth	/torso_cam3d_right/depth_registered	Registered depth corresponding to the right RGB image. Used to obtain the metric depth of detected objects.
Camera intrinsics	/torso_cam3d_left/rgb/camera_info, /torso_cam3d_right/rgb/camera_info	Provides camera intrinsic parameters required to back-project the detected pixel and its depth into a 3D point in the corresponding camera optical frame.
TF / robot pose	map, base_link, torso_cam3d_left_rgb_optical_frame, torso_cam3d_right_rgb_optical_frame	Provides the transformations required to convert camera-frame 3D detections into the global map frame and to obtain the robot pose.
Odometry	odom_combined	Configured odometry frame used by the SLAM/TF system.

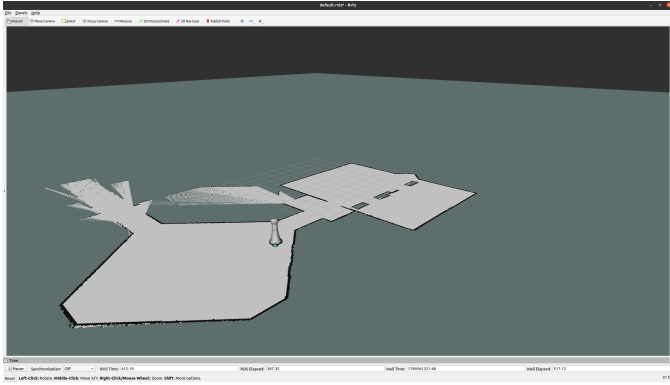


Fig. 2. LiDAR-based occupancy map generated using GMapping SLAM during exploration of the simulated indoor environment.

```
roslaunch teleop_twist_keyboard \
  teleop_twist_keyboard.py \
  cmd_vel:=/base/twist_mux/command_navigation
```

The generated map represents the geometric structure of the environment and provides the spatial reference required for placing detected privacy-sensitive objects.

D. Camera-Based Privacy Object Detection Layer

The second processing branch uses the camera mounted on the Care-O-Bot. Camera images are processed by a YOLO-based object detection node. The detector identifies privacy sensitive objects present in the camera image (see figure 3) and publishes the detection results through the /privacy_detections topic. Detections comprising the object class, confidence score, and bounding box coordinates are published.

The output of the detector contains image-level information about the detected objects. This information is not sufficient

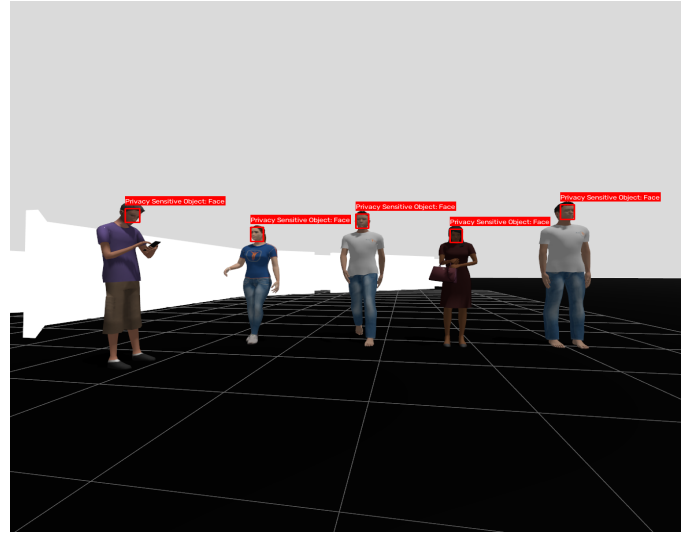


Fig. 3. Camera-based privacy-sensitive object detection using YOLO in the simulated environment.

by itself to determine the object's position on the environment map because image coordinates do not directly correspond to map coordinates.

Therefore, the detection output is passed to the object-position estimation stage.

E. Depth Transformation & Privacy Marker Generation Layer

Bounding boxes alone provide only image-plane coordinates. To determine the spatial location of a detected object (see figure 4), depth information is combined with the camera detection. The depth measurement provides the distance of the detected object relative to the camera. The resulting

three-dimensional position is initially expressed in the camera coordinate frame.

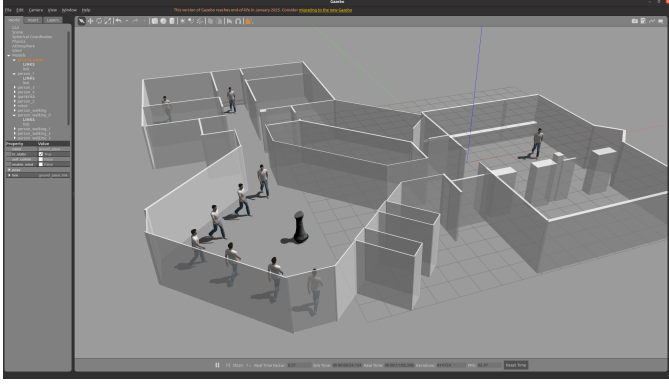


Fig. 4. Gazebo simulation environment showing the Care-O-Bot navigating a simulated indoor environment with privacy-sensitive objects.

Transforming the object position from the camera frame into the robot and map coordinate frames allows the detected object to be associated with a location in the environment map.

The node `object_position.py` associates each detection with the corresponding depth measurement and projects it into 3D camera coordinates using the camera intrinsics. The resulting point is then transformed from the camera optical frame into the global map frame using the TF tree, yielding a map-relative position for each detected object. These positions are published on `/privacy_objects_map`.

After the position of a detected privacy-sensitive object has been determined in the map coordinate system, the position is passed to the privacy marker component. The marker node publishes visualization information through the `/privacy_markers` topic.

`privacy_marker.py` converts each localised object into a `visualization_msgs/Marker` message and publishes it on `/privacy_markers`, allowing detected privacy-sensitive objects to be rendered directly on top of the occupancy grid (see figure 5).

RViz is used to display these markers together with the generated environment map. Consequently, the final visualization provides both the geometric structure of the environment and the locations of detected privacy-sensitive objects.

- **Map** (topic: `/map`)
- **LaserScan** (topic: `/scan_unified`)
- **RobotModel**
- **TF**
- **Marker** (topic: `/privacy_markers`)

This configuration produced the final combined output: a geometric occupancy map of the environment with privacy-sensitive objects marked at their estimated physical locations.

F. Object Detection and Privacy Marking Implementation

To identify and localize privacy-sensitive objects within the constructed environment map, the custom ROS package

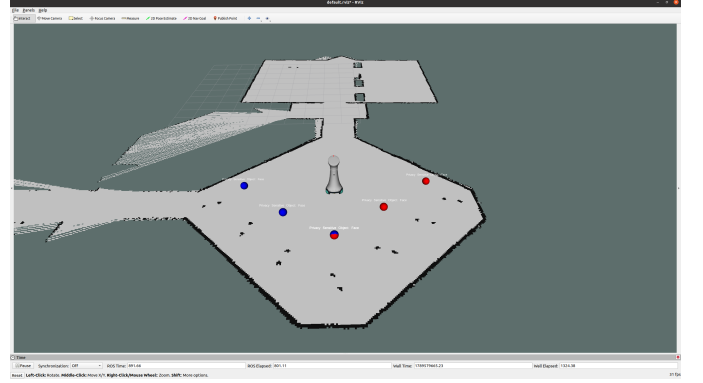


Fig. 5. RViz visualization of the generated occupancy map with the Care-O-Bot and localized privacy-sensitive objects marked at their estimated positions. Red markers indicate detections from the left torso camera, while blue markers indicate detections from the right torso camera.

`privacy_environment_map` implements three processing nodes for each of the left and right camera configurations: `privacy_detector.py`, `object_position.py`, and `privacy_marker.py`. The two camera pipelines operate in parallel, with each pipeline maintaining its own detection, localization, and visualization topics.

- 1) **privacy_detector.py:** Performs privacy-sensitive object detection using the YOLO model `Model.pt`. Two instances of the detector are executed, one with `_camera_id:=left` and one with `_camera_id:=right`. Each detector processes its corresponding camera stream and publishes the detected object class, confidence, bounding-box coordinates, and detection timestamp for subsequent localization.
- 2) **object_position.py:** Receives object detections from the privacy detector and estimates their 3D positions in the global map frame. For each detection, the bounding-box centre is combined with the corresponding metric depth from the registered depth camera and back-projected into the camera optical frame using the camera intrinsics. The resulting 3D point is transformed directly into the map frame using the robot TF tree. Separate configurations are maintained for the left and right cameras, and persistent track IDs allow multiple detected objects to be localized independently.
- 3) **privacy_marker.py:** Receives the estimated map positions from the corresponding `object_position.py` instance and visualizes them in RViz using ROS Marker messages. Separate marker nodes are executed for the left and right camera configurations. Each tracked object is assigned a persistent marker ID, and both a spherical marker and a text label are published for visualization. Markers are maintained persistently and are explicitly deleted when the corresponding object track becomes stale.

The complete system therefore consists of two parallel camera-specific processing pipelines. The camera-specific in-

stances are launched using the `_camera_id` parameter, while unique ROS node names are assigned to the localization and marker nodes. This organization allows detections from the left and right camera configurations to be processed independently while using the common SLAM-generated map frame for global localization.

G. ROS Communication

The implemented system is organized as a set of independent ROS nodes that communicate through topics. Two parallel processing pipelines are used for the left and right torso-mounted cameras. The principal ROS topics used by the system are summarized in Table II.

H. Final System Output

The final output of the system is an environment map containing visual markers corresponding to detected privacy-sensitive objects (see figure 6). The map therefore contains both spatial information obtained from LiDAR-based SLAM and semantic privacy information obtained from camera-based object detection.

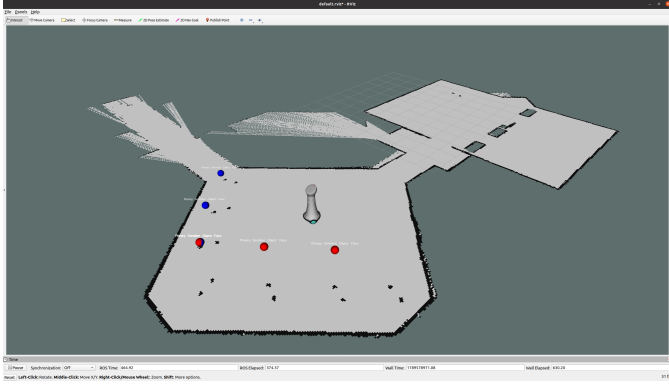


Fig. 6. Final privacy-aware environment map in RViz, showing the LiDAR-based occupancy map, Care-O-Bot pose, and localized privacy-sensitive objects. Red markers denote left-camera detections, while blue markers denote right-camera detections.

V. EXPERIMENTAL RESULTS

This section presents the results obtained at each stage of the pipeline: map construction, object detection, and the final combined privacy-aware map.

A. Experimental Setup

All experiments were conducted in simulation using Gazebo, with the Care-O-Bot 4 (cob4-5) model deployed in the `ipa-apartment` based environment. The environment consists of a set of connected corridors and office rooms containing walls, doorways, and furniture. The robot was driven manually via keyboard teleoperation during mapping. Table III summarises the configuration used.

B. Mapping Results

The GMapping-based SLAM pipeline successfully produced a coherent 2D occupancy grid map of the simulated laboratory environment. The map correctly represents navigable corridors and obstacle boundaries as detected by the front-facing LiDAR sensor. Map generation was stable across multiple teleoperation runs.

C. Detection Results

The YOLOv8n model, running in inference-only mode on the pretrained COCO weights, successfully detected privacy-relevant object categories in test images with high confidence. Detection latency was within real-time bounds, confirming the feasibility of the detection pipeline for integration with a live robot feed.

D. Final Environment Map Results

The complete pipeline functions end-to-end: the robot successfully constructs a 2D occupancy map of the office environment, detects privacy-sensitive objects from its camera feed, localises them in 3D, and renders them as persistent markers within the map frame (see figure 7 and 8).

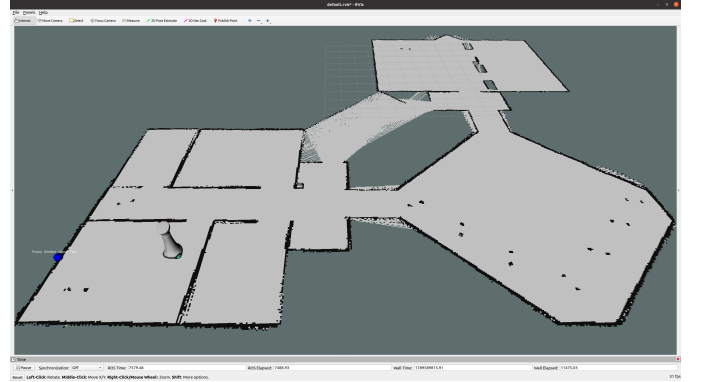


Fig. 7. RViz visualization of the integrated privacy-aware mapping system, showing the LiDAR-generated occupancy grid together with the Care-O-Bot pose and detected privacy-sensitive objects localized in the global map frame.

VI. FUTURE WORK

While the implemented system demonstrates a complete end-to-end privacy-aware mapping pipeline, several avenues remain for improvement and extension.

A. Improved Mapping and Localisation

The current system relies on `gmapping`, a particle-filter SLAM approach that produces a purely 2D occupancy grid. Although sufficient for a structured indoor environment, this representation discards vertical information, meaning that objects at different heights collapse onto the same map cell. Future work could adopt a graph-based SLAM system such as `Cartographer`, which offers loop-closure detection and typically yields greater global consistency over longer trajectories.

TABLE II
MAIN ROS TOPICS USED IN THE IMPLEMENTED PRIVACY-SENSITIVE OBJECT DETECTION AND LOCALIZATION SYSTEM.

Topic	Purpose
/privacy_detections_left	Detection results from the left RGB camera, including object class, confidence, bounding box, timestamp, and camera identifier
/privacy_detections_right	Detection results from the right RGB camera, including object class, confidence, bounding box, timestamp, and camera identifier
/privacy_objects_map_left	Left-camera object positions transformed into the global map frame
/privacy_objects_map_right	Right-camera object positions transformed into the global map frame
/privacy_markers_left	RViz visualization markers for left-camera privacy-sensitive objects
/privacy_markers_right	RViz visualization markers for right-camera privacy-sensitive objects

TABLE III
EXPERIMENTAL CONFIGURATION.

Parameter	Value
Robot model	Care-O-Bot 4 (cob4-5)
Environment	ipa-apartment
Simulator	Gazebo
Middleware	ROS Noetic
SLAM algorithm	gmapping (RBPF)
Detector	YOLO (custom weights, Model.pt)

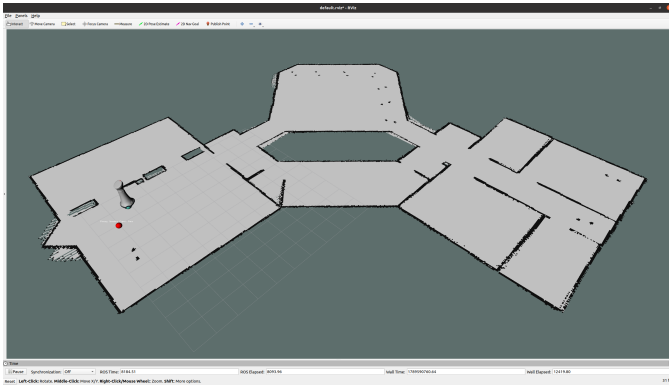


Fig. 8. RViz visualization demonstrating the spatial localization of privacy-sensitive objects within the LiDAR-based environment map.

Alternatively, a better 3D mapping representation such as an OctoMap could be used.

Additionally, particle depletion was observed during sharp turns, suggesting that the odometry motion model could be better tuned to the Care-O-Bot's omnidirectional base, which does not match the differential-drive assumptions underlying gmapping's default noise parameters.

B. Autonomous Exploration

Mapping in this project was performed through manual keyboard teleoperation. Replacing this with an autonomous exploration strategy, for example, frontier-based exploration combined with the ROS navigation stack would allow the robot to build a privacy-aware map of an unknown environment without human intervention, which is a prerequisite for practical deployment.

C. Enhanced Detection and Object Classes

The detection component currently uses a single YOLO model with a fixed set of privacy-sensitive classes. Several improvements are possible:

- **Expanded and refined training data**, covering a broader range of privacy-relevant objects and a wider variety of viewing angles, lighting conditions, and occlusions.
- **Confidence thresholding and temporal filtering**, so that transient false positives from a single frame do not result in spurious permanent markers on the map.
- **Multi-view fusion and object tracking**, associating repeated detections of the same physical object across consecutive frames to produce a single, more accurately localised marker rather than a cluster of near-duplicate entries.
- **Graded privacy levels**, replacing a binary sensitive/not-sensitive classification with a severity ranking, allowing downstream behaviours to respond proportionately.

D. Improved Localisation Accuracy

Object positions are presently derived from a single depth sample associated with each bounding box. Robustness could be improved by aggregating depth measurements across the bounding box region (for example, using a median or segmentation-based mask to reject background pixels), and by propagating uncertainty from the depth measurement and the robot's pose estimate into a covariance associated with each marked object.

E. Acting on the Privacy Map

At present the system *records* privacy-sensitive locations but does not act upon them. A natural extension is to use the privacy map to influence robot behaviour, such as:

- Generating costmap layers that discourage navigation through privacy-sensitive regions.
- Automatically disabling, blurring, or discarding camera data when the robot is oriented towards a marked object.
- Restricting data logging or teleoperation video streaming in sensitive areas.

F. Validation on Real Hardware

All experiments were conducted in simulation, where sensor noise, lighting variation, and odometry drift are considerably

more benign than in reality. Deploying the pipeline on a physical Care-O-Bot, or a comparable mobile platform, would be necessary to assess real-world robustness particularly the performance of the detector under realistic lighting and the accuracy of depth-based localisation with a real RGB-D sensor.

G. Quantitative Evaluation

Finally, the present evaluation is largely qualitative. A more rigorous assessment would compare the generated map against the known ground-truth geometry of the Gazebo world, and compare estimated object positions against their true simulated coordinates, reporting metrics such as mean localisation error, detection precision and recall, and map accuracy.

VII. CONCLUSION

This paper presented an approach for creating a privacy-aware environment map using the Care-O-Bot 4 service robot. First, familiarity with the Care-O-Bot platform and its simulator was established. Second, a 2D occupancy grid map of the environment was constructed from LiDAR data using the gmapping SLAM package. Third, privacy-sensitive objects were identified from the robot's torso-mounted RGB camera using a YOLO-based detector. Fourth, these detections were converted from image-plane bounding boxes into map-frame coordinates using the depth camera and the robot's transform tree, and subsequently rendered as persistent markers overlaid on the occupancy grid (environment map) in RViz. The result is a single unified representation combining geometric and privacy-relevant semantic information.

The principal outcome of this work is a functioning end-to-end pipeline demonstrating that privacy awareness can be embedded directly into a robot's spatial model of its environment, rather than treated as a separate perception output. The system demonstrates the feasibility of combining standard SLAM and deep learning detection approaches to address the emerging problem of spatial privacy awareness in service robotics.

The system nonetheless remains a proof of concept. It was evaluated only in simulation, relies on manual exploration, and records privacy-sensitive locations without acting upon them. Addressing these limitations, as outlined in Section VI, would move the approach closer to practical deployment on robots operating in genuinely private, human-occupied spaces.

ACKNOWLEDGEMENT

We sincerely thank our course coordinators, Dr. Delphine Reinhardt and Prof. Sebastian Reinke, for their guidance, valuable advice and feedback throughout the project. Their support was instrumental in successfully completing this work.

REFERENCES

- [1] S. Thrun, W. Burgard, and D. Fox, *Probabilistic Robotics*, MIT Press, 2005.
- [2] F. A. Moreno, J. L. Blanco, and J. González, "A Kalman filter implementation for precision improvement in low-cost GPS positioning of autonomous vehicles," *Sensors*, vol. 16, no. 1, 2016.
- [3] G. Grisetti, C. Stachniss, and W. Burgard, "Improved Techniques for Grid Mapping With Rao-Blackwellized Particle Filters," *IEEE Transactions on Robotics*, vol. 23, no. 1, pp. 34–46, 2007.

- [4] Open Source Robotics Foundation, *gmapping: slam_gmapping*, ROS Documentation.
- [5] W. Hess, D. Kohler, H. Rapp, and D. Andor, "Real-Time Loop Closure in 2D LIDAR SLAM," *2016 IEEE International Conference on Robotics and Automation (ICRA)*, pp. 1271–1278, 2016.
- [6] R. Mur-Artal, J. M. M. Montiel, and J. D. Tardos, "ORB-SLAM: A Versatile and Accurate Monocular SLAM System," *IEEE Transactions on Robotics*, vol. 31, no. 5, pp. 1147–1163, 2015.
- [7] J. Redmon, S. Divvala, R. Girshick, and A. Farhadi, "You Only Look Once: Unified, Real-Time Object Detection," *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, pp. 779–788, 2016.
- [8] G. Jocher, A. Chaurasia, and J. Qiu, *Ultralytics YOLOv8*, version 8.0.0, 2023.
- [9] S. Ren, K. He, R. Girshick, and J. Sun, "Faster R-CNN: Towards Real-Time Object Detection with Region Proposal Networks," *Advances in Neural Information Processing Systems (NeurIPS)*, vol. 28, 2015.
- [10] R. F. Salas-Moreno, R. A. Newcombe, H. Strasdat, P. H. J. Kelly, and A. J. Davison, "SLAM++: Simultaneous localisation and mapping at the level of objects," in *Proc. IEEE Conf. Computer Vision and Pattern Recognition (CVPR)*, 2013, pp. 1352–1359.
- [11] S. Birchfield and K. Konolige, "Uncertainty in robotics navigation and its implication for system design," *IEEE Robotics Automation Mag.*, 2007.
- [12] A. Das, P. Bhatt, and K. Nair, "Privacy-aware robot navigation in human environments," in *Proc. Int. Conf. Robot and Human Interactive Communication (RO-MAN)*, 2021.
- [13] I. Kostavelis and A. Gasteratos, "Semantic mapping for mobile robotics tasks: A survey," *Robotics and Autonomous Systems*, vol. 66, pp. 86–103, 2015.
- [14] M. Rueben, A. M. Aroyo, C. Lutz, J. Schmölz, P. Van Cleynenbreugel, A. Corti, S. Agrawal, and W. D. Smart, "Themes and Research Directions in Privacy-Sensitive Robotics," *2018 IEEE Workshop on Advanced Robotics and its Social Impacts (ARSO)*, pp. 77–84, 2018.
- [15] U. Reiser, C. Connette, J. Fischer, J. Kubacki, A. Bubeck, F. Weisshardt, T. Jacobs, C. Parltitz, M. Hägele, and A. Verl, "Care-O-bot 3 – Creating a product vision for a utility robot for everyday use in private home environments," in *Proc. IEEE/RSJ Int. Conf. Intelligent Robots and Systems (IROS)*, 2009, pp. 5972–5978.
- [16] N. Sünderhauf, O. Brock, W. Scheirer, R. Hadsell, D. Fox, J. Leitner, B. Upcroft, P. Abbeel, W. Burgard, M. Milford, and P. Corke, "The limits and potentials of deep learning for robotics," *Int. J. Robotics Research*, vol. 37, no. 4–5, pp. 405–420, 2018.
- [17] T. Lin, M. Maire, S. Belongie, J. Hays, P. Perona, D. Ramanan, P. Dollár, and C. L. Zitnick, "Microsoft COCO: Common objects in context," in *Proc. European Conf. Computer Vision (ECCV)*, 2014, pp. 740–755.

AI USAGE DECLARATION

In accordance with the course requirements, we disclose the following regarding AI tool usage in this work:

- 1) **Which AI tools were used?** ChatGPT and Claude were used as an AI assistant during this project.
- 2) **For what purpose were AI tools used?** AI was used for: assistance in debugging ROS configuration issues, guidance on YOLOv8 integration with ROS, and support in structuring and drafting this report.
- 3) **Which parts of the work were produced with AI assistance?** AI tools were employed as a supplementary resource for resolving technical problems in the ROS setup, understanding the object-position estimation and localization approach, and improving the organization of the project. The report text was corrected with AI assistance.
- 4) **How were AI outputs verified?** All AI-suggested ROS commands and configurations were tested on the live lab system. Commands that produced errors or incorrect

behavior were debugged manually. The AI did not have direct access to the lab system.

- 5) **What are the limitations of the AI assistance received?** The AI assistant operated without access to the actual robot system and could not verify the correctness of outputs against live sensor data. Several AI suggestions required correction after testing on the real system.